

Compito di Programmazione - Bioinformatica

9 settembre 2025 (tempo disponibile: 2 ore)

(si consegnino i file `sentence.h` e `sentence.c` opportunamente completati)

La struttura `sentence` implementa una frase, cioè una lista di parole. È definita dentro `sentence.h`:

```
struct sentence {  
    char *word;  
    struct sentence *next;  
};
```

Una frase può essere costruita a partire da un array di stringhe (`construct_sentence`) e successivamente deallocata (`free_sentence`). Inoltre se ne può calcolare la lunghezza, cioè il numero di caratteri totale delle sue parole, contando anche uno spazio tra una parola e l'altra (`length_sentence`). Infine, una frase può essere trasformata in una nuova stringa, ottenuta concatenando le sue parole e aggiungendo uno spazio tra una parola e l'altra (`concat_sentence`). La stessa operazione può essere effettuata in senso inverso, cioè iniziando, nella concatenazione, dall'ultima parola e terminando con la prima parola (`inverse_concat_sentence`).

Il seguente `main.c`, già fatto, completo e da non modificare, esemplifica l'utilizzo di una frase e delle sue funzioni citate sopra:

```
#include <stdio.h>  
#include <stdlib.h>  
#include "sentence.h"  
  
int main(void) {  
    char *words[] = {  
        "Nel", "mezzo", "del", "cammin", "di", "nostra", "vita"  
    };  
  
    struct sentence *s = construct_sentence(words, 7);  
    printf("La lunghezza della frase e' %i\n", length_sentence(s));  
    char *concat = concat_sentence(s);  
    printf("La sua concatenazione e' \"%s\"\n", concat);  
    free(concat);  
    concat = inverse_concat_sentence(s);  
    printf("La sua concatenazione inversa e' \"%s\"\n", concat);  
    free(concat);  
    free_sentence(s);  
  
    return 0;  
}
```

Quando avrete implementato tutte le funzioni correttamente, l'esecuzione del `main` dovrà stampare:

```
La lunghezza della frase e' 35  
La sua concatenazione e' "Nel mezzo del cammin di nostra vita"  
La sua concatenazione inversa e' "vita nostra di cammin del mezzo Nel"
```

Esercizio 1 (1 punto)

Si completi il file `sentence.h` aggiungendo la dichiarazione delle cinque funzioni `construct_sentence`, `free_sentence`, `length_sentence`, `concat_sentence` e `inverse_concat_sentence`. Dentro `sentence.c` trovate la bozza di tali funzioni.

Esercizio 2 (4 punti)

Si completi la funzione `construct_sentence` dentro `sentence.c`. Si noti che tale funzione copia, dentro la frase risultante, il puntatore alle stringhe passate come argomento, quindi non fa una copia di tali stringhe.

Esercizio 3 (3 punti)

Si completi la funzione `free_sentence` dentro `sentence.c`. Si noti che tale funzione deve deallocare tutta la lista, precedentemente allocata con `construct_sentence`, non solo il suo primo elemento.

Esercizio 4 (5 punti)

Si completi la funzione `length_sentence` dentro `sentence.c`. Si noti che tale funzione, nel calcolare la lunghezza, considera uno spazio tra una parola e l'altra delle frase.

Esercizio 5 (8 punti)

Si completi la funzione ausiliaria `concat_sentence_rec` dentro `sentence.c`, che concatena le parole di una frase dentro una stringa risultato, con uno spazio tra una parola e l'altra. **Tale funzione deve essere ricorsiva.** La si usi quindi per completare la funzione `concat_sentence` dentro `sentence.c`. **Suggerimento:** in `concat_sentence_rec` si considerino i casi: frase vuota, frase di una sola parola, frase di più parole.

Esercizio 6 (10 punti)

Si completi la funzione ausiliaria `inverse_concat_sentence_rec` dentro `sentence.c`, che concatena le parole di una frase dentro una stringa risultato, con uno spazio tra una parola e l'altra, in ordine inverso, cioè mettendo all'inizio l'ultima parola e alla fine la prima parola. **Tale funzione deve essere ricorsiva.** La si usi quindi per completare la funzione `inverse_concat_sentence` dentro `sentence.c`. **Suggerimento:** in `inverse_concat_sentence_rec` si considerino i casi: frase vuota, frase di una sola parola, frase di più parole.